CUET PG Computer Science / MCA Entrance

Digital Fundamentals & Digital Logic Design

Master Study Notes & Comprehensive Revision Guide (Lectures 01 to 17)

* Comprehensive coverage of all 17 lectures & core Digital Fundamentals units
* Explained in beginner-friendly Hinglish with step-by-step logic and intuition
* Complete logic gate circuit diagrams, truth tables, and K-Map simplifications
* Meticulously solved step-by-step calculations, formula sheets & exam traps
* Designed specifically for CUET PG, MCA, and State Entrance Examinations

[1] Master Conceptual Notes

All 17 Units: Number Systems, Complements, Floating Point, Boolean Algebra, Gates, K-Maps, Adders/ Subtractors, MUX, Latches, Flip-Flops, Registers, Counters & Memory Hierarchy.

[2] Exam-Oriented Focus

Detailed error traps, quick revision formulas, difference tables, step-by-step solved numericals, and CUET PG expected questions with complete Hinglish explanations.

Target: CUET PG CS/MCA, NIMCET, GATE & University Exams

Material Type: Complete Theory, Solved Examples, Diagrams & Revision Sheets
Syllabus Coverage: Complete 17 Units (Lectures 01 to 17)
Language: Easy Hinglish (Bilingual Hindi + English explanation for rapid conceptual mastery)
Edition: Updated Exam Edition

TABLE OF CONTENTS & COURSE SYLLABUS MAPPING

Ye master study note CUET PG Computer Science / MCA ke complete syllabus aur PhysicsWallah Samarth batch ki 17 digital fundamental lectures ko systematically beginner-friendly Hinglish mein cover karta hai. Original class notes mein jitne bhi calculation errors, blank missing pages (jaise Lecture 14), aur incomplete notes the, unhe verify karke standard textbooks (Morris Mano & Floyd) ke according accurately rectify kiya gaya hai.

Unit	Lecture / Topic Name	Key Concepts Covered
Unit 01	Intro to Digital Systems & Data Units	Analog vs Digital, Bits, Nibbles, Bytes, KB vs KiB, Powers of 2
Unit 02	Number Systems & Base Conversions	Decimal, Binary, Octal, Hexadecimal, Base-r, 3-bit & 4-bit grouping
Unit 03	Complements & Fixed-Point Numbers	r's & (r-1)'s, 1's & 2's, Signed Magnitude, Range, Overflow detection
Unit 04	Floating Point (IEEE 754 Standard)	Single (32-bit) & Double (64-bit), Bias 127/1023, Special values (+inf, NaN)
Unit 05	Computer Arithmetic & Digital Codes	Binary operations, Booth's Algorithm, BCD, Excess-3, Gray Code
Unit 06	Digital Computers & CPU Organization	PC, IR, MAR, MDR, AC registers, Buses, Control Unit, Instruction cycle
Unit 07	Boolean Algebra Axioms & Laws	Postulates, Idempotent, Distributive, Absorption, De Morgan, Consensus
Unit 08	Logic Gates & Universal Realization	AND, OR, NOT, NAND, NOR, XOR, XNOR, Min Universal Gate Count table
Unit 09	Standard & Canonical Forms (SOP/POS)	Minterms (m), Maxterms (M), Sigma and Pi notation, Truth table forms
Unit 10	Karnaugh Maps (K-Maps) Part 1	Gray code layout, 2 & 3-variable maps, Grouping, PIs and EPIs
Unit 11	Karnaugh Maps (K-Maps) Part 2	4-variable maps, 4-corner quad, Don't cares (d), POS minimization
Unit 12	Combinational Circuits: Adders/Subtractors	Half & Full Adders, Half & Full Subtractors, Carry Lookahead Adder
Unit 13	Multiplexers, Decoders & Encoders	2:1, 4:1, 8:1 MUX, DEMUX, n-to-2^n Decoders, Priority Encoders
Unit 14	Sequential Circuits & Latches	Combinational vs Sequential, Latches, SR latch, Triggering, Setup/Hold
Unit 15	Flip-Flops & Conversions	SR, JK, Race-Around, Master-Slave, D, T, Characteristic & Excitation
Unit 16	Registers & Counters	SISO, SIPO, PISO, PIPO, Ring & Johnson counters, Modulo-N counters
Unit 17	Memory Unit, Cache & CUET PG PYQs	Hierarchy, SRAM vs DRAM, Cache mapping, Hit ratio, CUET PG PYQs

UNIT / LECTURE 01

Introduction to Digital Systems & Data Units

Topic Overview: Digital vs Analog concepts, Binary digits, Bits, Nibbles, Bytes, Data units (KB, MB, GB, KiB, MiB) aur computer data representation ka foundation.

1. Analog versus Digital Systems

Analog aur Digital signals mein kya basic difference hai?

Digital Fundamentals computer science ka core subject hai jo explain karta hai ki computer hardware information ko kaise store aur process karta hai.

Analog Signal ek continuous signal hota hai jo time ke har instant par continuous values leta hai (jaise sound waves, mercury thermometer ka mercury level, ya natural speech). Isme infinite possible values hoti hain.

Digital Signal ek discrete signal hota hai jo sirf specific discrete voltage levels ko represent karta hai. Modern binary digital computers mein sirf 2 distinct logic states use hoti hain: Logic 0 (Low Voltage / OFF, approx 0V) aur Logic 1 (High Voltage / ON, approx +3.3V ya +5V).

Advantage of Digital Systems: Digital circuits mein noise immunity bohot high hoti hai (chhota mota voltage noise hone par bhi 0 aur 1 correctly detect ho jata hai), data storage permanent aur error-free hota hai, aur logic design programmable hota hai.

Parameter	Analog Signal	Digital Signal
Signal Nature	Continuous with time	Discrete values (0 and 1)
Values Possible	Infinite values within a range	Finite discrete levels (Binary: 2 levels)
Noise Immunity	Very Low (Noise easily distorts signal)	Very High (Noise does not easily flip 0 to 1)
Storage & Processing	Difficult to store without loss	Extremely easy & accurate using flip-flops/ RAM
Examples	Human voice, Mercury Thermometer	Computer CPU, USB Flash Drive, Digital Clock

[MUST REMEMBER FOR CUET PG] Analog vs Digital in CUET PG

Exam mein direct question pucha jata hai: "Which signal is discrete in nature and immune to noise?" -> Answer: Digital Signal. Digital signals square wave form mein hote hain jabki analog signals continuous sine/arbitrary waveform mein hote hain.

2. Units of Digital Data (Bit, Nibble, Byte & Higher Units)

Computer memory aur data capacity measurement units

Bit (Binary Digit): Computer data ki sabse smallest unit hoti hai. Ek single bit ki value ya to 0 hoti hai ya 1.

Nibble: 4 consecutive bits ke group ko 1 Nibble kehte hain. (1 Nibble = 4 Bits). Example: 1011 ek nibble hai. 1 Hexadecimal digit exactly 1 Nibble ko represent karta hai!

Byte: 8 consecutive bits ke group ko 1 Byte kehte hain. (1 Byte = 8 Bits = 2 Nibbles). Computer architecture mein standard addressable memory unit 1 Byte hoti hai.

Word: CPU architecture ke hisaab se word size define hoti hai. 32-bit CPU ka word size 32 bits (4 bytes) hota hai, aur 64-bit CPU ka word size 64 bits (8 bytes) hota hai.

Unit Name	Symbol	Binary Size (Power of 2)	Decimal Standard (SI)
Bit	b	1 binary digit (0 or 1)	Single binary digit
Nibble	-	4 bits	Half byte
Byte	B	8 bits	8 bits
Kilobyte / Kibibyte	KB / KiB	1,024 Bytes (2^10 Bytes)	1,000 Bytes (10^3) in storage industry
Megabyte / Mebibyte	MB / MiB	1,048,576 Bytes (2^20 Bytes)	10^6 Bytes (1,000,000 Bytes)
Gigabyte / Gibibyte	GB / GiB	1,073,741,824 Bytes (2^30 Bytes)	10^9 Bytes
Terabyte / Tebibyte	TB / TiB	2^40 Bytes	10^12 Bytes
Petabyte / Pebibyte	PB / PiB	2^50 Bytes	10^15 Bytes
Exabyte / Exbibyte	EB / EiB	2^60 Bytes	10^18 Bytes

[EXAM TRAP / COMMON CONFUSION] KB vs Kb (Capital B vs Small b)

* Capital B = Byte (8 bits), Small b = bit.
* 1 KB = 1 Kilobyte = 8 Kilobits (8 Kb).
* Data transfer speed (e.g. Internet 100 Mbps) small b (megabits per second) mein hoti hai! Storage capacity (e.g. 100 MB) capital B (megabytes) mein hoti hai.
* If asked: How many bits in 10 KB? -> 10 * 1024 * 8 = 81,920 bits.

[SOLVED EXAMPLE] Data Unit Conversion (Bits to KB)

Problem: Calculate how many Kilobytes (KB) are present in 10 Megabits (Mb).

Step-by-Step Solution:

Step 1: Given value = 10 Megabits (Mb). Notice "b" is small, so it represents bits.

Step 2: Step 1: Convert Megabits to bits: 10 Mb = 10 * 1,000,000 bits (or 10 * 2^20 bits in binary).

Step 3: In standard CS binary calculation: 10 * 2^20 bits = 10,485,760 bits.

Step 4: Step 2: Convert bits to Bytes: Divide by 8 -> 10,485,760 / 8 = 1,310,720 Bytes.

Step 5: Step 3: Convert Bytes to Kilobytes (KB): Divide by 1024 -> 1,310,720 / 1024 = 1,280 KB.

Step 6: (Note: Agar decimal SI system 10^3 use karein: 10 * 10^6 / 8 = 1,250,000 Bytes / 1000 = 1250 KB).

Final Result: 1280 KB (Binary / IEC) or 1250 KB (SI Decimal).

UNIT / LECTURE 02

Number Systems & Base Conversions

Topic Overview: Positional number systems: Decimal (10), Binary (2), Octal (8), Hexadecimal (16), Base-r representation, Integer & Fractional conversions, aur Fast 3-bit/4-bit grouping shortcuts.

1. Positional Number Representation & Radix (Base)

Number system ka general formula aur place values

Har number system ka ek base ya radix (r) hota hai. Radix r represent karta hai ki us system mein total kitne unique symbols/digits use hote hain.

General Positional Formula: Kisi bhi base-r number (d_n d_{n-1} ... d_0 . d_{-1} d_{-2} ...)_r ki decimal value hoti hai:

Value = d_n * r^n + d_{n-1} * r^{n-1} + ... + d_0 * r^0 + d_{-1} * r^{-1} + d_{-2} * r^{-2} + ...

Char standard number systems computer science mein use hote hain:

1. Decimal System (Base 10): Digits {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}. Weight = powers of 10.

2. Binary System (Base 2): Digits {0, 1}. Weight = powers of 2. Internal hardware representation.

3. Octal System (Base 8): Digits {0, 1, 2, 3, 4, 5, 6, 7}. 1 octal digit exactly 3 binary bits ke barabar hota hai (2^3 = 8).

4. Hexadecimal System (Base 16): Digits {0 to 9, A=10, B=11, C=12, D=13, E=14, F=15}. 1 hex digit exactly 4 binary bits ke barabar hota hai (2^4 = 16).

Decimal	Binary (4-bit)	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2. Conversion: Decimal to Any Base & Any Base to Decimal

Repeated Division/Multiplication method aur Polynomial expansion

Decimal -> Any Base r: Integer part ko repeatedly r se divide karo aur remainders ko reverse order (bottom-to-top / LSB to MSB) mein likho. Fractional part ko repeatedly r se multiply karo aur integer carry part ko top-to-bottom order mein likho.

Any Base r -> Decimal: Har digit ko uske place value (r^k) se multiply karke saare terms ko add kar do.

[EXAM TRAP / COMMON CONFUSION] Important Correction from Class Slide

* PW Class Slide Error Alert: Original slide note mein example tha: (0111 1111 0011)_2 -> Hex conversion. Slide mein galti se 8F3_16 likh diya gaya tha!
* Correct calculation: 0111 = 7, 1111 = F, 0011 = 3. Final answer is 7F3_16! Exam mein aisi silly mistakes bilkul na karein.

[SOLVED EXAMPLE] Decimal to Binary (Integer + Fraction)

Problem: Convert (56.70)_10 into binary up to 4 fractional places.

Step-by-Step Solution:

Step 1: Integer Part (56): Repeated division by 2:
Step 2: 56 / 2 = 28, Remainder = 0
Step 3: 28 / 2 = 14, Remainder = 0
Step 4: 14 / 2 = 7, Remainder = 0
Step 5: 7 / 2 = 3, Remainder = 1
Step 6: 3 / 2 = 1, Remainder = 1
Step 7: 1 / 2 = 0, Remainder = 1
Step 8: Reading bottom to top: (56)_10 = (111000)_2.
Step 9: Fractional Part (0.70): Repeated multiplication by 2:
Step 10: 0.70 * 2 = 1.40 -> Integer part = 1, remaining fraction = 0.40
Step 11: 0.40 * 2 = 0.80 -> Integer part = 0, remaining fraction = 0.80
Step 12: 0.80 * 2 = 1.60 -> Integer part = 1, remaining fraction = 0.60
Step 13: 0.60 * 2 = 1.20 -> Integer part = 1, remaining fraction = 0.20
Step 14: Reading top to bottom: (0.70)_10 = (0.1011...)_2.

Final Result: (56.70)_10 = (111000.1011...)_2

3. Fast Grouping Method: Binary <-> Octal and Binary <-> Hexadecimal

No need to go through decimal! Direct grouping trick

Binary to Octal: Binary number ko Radix point se dono taraf 3-3 bits ke groups mein baanto. Left integer part mein left side zero add kar sakte hain, fractional part mein right side zero add kar sakte hain.

Binary to Hexadecimal: Radix point se dono taraf 4-4 bits ke groups banao. Har group ko uske corresponding Hex digit (0-9, A-F) se replace karo.

Octal to Hexadecimal: Pehle Octal digits ko 3-bit binary mein convert karo, phir pure binary stream ko 4-bit groups mein regroup karke Hex digit banao!

[SOLVED EXAMPLE] Octal to Hexadecimal via Binary

Problem: Convert (746.23)_8 into Hexadecimal.

Step-by-Step Solution:

Step 1: Step 1: Har octal digit ko 3-bit binary mein likho:

Step 2: 7 -> 111, 4 -> 100, 6 -> 110, 2 -> 010, 3 -> 011

Step 3: Binary equivalent: (111100110.010011)_2

Step 4: Step 2: Radix point ke left aur right 4-bit groups banao:

Step 5: Integer (left): 111100110 -> [0001] [1110] [0110] (padded left with 000)

Step 6: [0001] = 1, [1110] = 14 = E, [0110] = 6 -> Integer = 1E6

Step 7: Fraction (right): .010011 -> [0100] [1100] (padded right with 00)

Step 8: [0100] = 4, [1100] = 12 = C -> Fraction = .4C

Final Result: (746.23)_8 = (1E6.4C)_16

UNIT / LECTURE 03

Complements & Fixed-Point Signed Representations

Topic Overview: Radix complement (r's complement) aur Diminished radix complement ((r-1)'s complement), 1's & 2's complement, Signed Magnitude, Range formulas, aur Overflow detection rules.

1. Complements in Number Systems: r's and (r-1)'s Complements

Complements kyu use hote hain aur calculate kaise karte hain?

Digital circuits mein subtraction perform karne ke liye separate subtractor circuit banane ke bajaye adder circuit hi use kiya jata hai! Subtraction A - B ko A + (Complement of B) ke form mein convert kar diya jata hai.

Har base r mein do tarah ke complements hote hain:

1. Diminished Radix Complement ((r-1)'s Complement): Base 10 mein 9's complement, Base 2 mein 1's complement, Base 8 mein 7's complement, Base 16 mein 15's (F's) complement.

Formula for integer with n digits: (r-1)'s complement = (r^n - 1) - N

Binary mein 1's complement nikalna super easy hai: Sirf saare bits ko flip/invert kar do (0 ko 1, 1 ko 0)!

2. Radix Complement (r's Complement): Base 10 mein 10's complement, Base 2 mein 2's complement.

Formula: r's complement = (r-1)'s complement + 1

Binary mein 2's complement = 1's complement + 1.

[IMPORTANT FORMULA / LAW] Shortcut for 2's Complement

* Direct Inspection Shortcut: Number ko Right se Left scan karo. Pehle '1' tak ke saare bits (including that first 1) as-it-is copy karo. Uske baad ke bache hue saare left bits ko invert (flip) kar do!
Example: N = 1011000
Right se pehla 1: ...1000 (same rahega).
Left bits 101 -> flip ho jayenge 010.
Result = 0101000. Exactly 2's complement!

2. Signed Binary Number Representations

Unsigned, Signed Magnitude, 1's Complement, aur 2's Complement

Computers positive aur negative dono numbers ko store karte hain. Most Significant Bit (MSB) hamesha SIGN BIT hota hai: 0 = Positive (+), 1 = Negative (-).

A. Signed Magnitude: MSB sign bit hota hai, baaki ke (n-1) bits magnitude (value) represent karte hain. Disadvantage: Do alag representations hote hain zero ke (+0 = 0000, -0 = 1000).

B. 1's Complement: Positive number as-it-is binary mein hota hai. Negative number uske magnitude ka 1's complement hota hai. Disadvantage: Yahan bhi double zero problem hai (+0 = 0000, -0 = 1111).

C. 2's Complement (Standard in Modern Computers): Positive number standard binary hota hai (MSB = 0). Negative number magnitude ka 2's complement hota hai (MSB = 1).

Advantage of 2's Complement: Zero ka sirf EK unique representation hota hai (00000000). Arithmetic circuits super simple ho jate hain!

Representation (n bits)	Minimum Value	Maximum Value	Zero Representation
Unsigned Binary	0	2^n - 1	Single: 000...0
Signed Magnitude	-(2^(n-1) - 1)	+(2^(n-1) - 1)	Two zeros: +0 (000..) and -0 (100..)
1's Complement	-(2^(n-1) - 1)	+(2^(n-1) - 1)	Two zeros: +0 (000..) and -0 (111..)
2's Complement	-2^(n-1)	+(2^(n-1) - 1)	Single unique zero: 000...0

[MUST REMEMBER FOR CUET PG] CUET PG High-Frequency Question: Range of n-bit 2's Complement

* 8-bit 2's complement range: -2^7 to +(2^7 - 1) = -128 to +127.
* Notice: Negative side mein ek extra number (-128 = 10000000) represent ho sakta hai kyunki zero ka sirf ek form hota hai!
* 16-bit range: -32,768 to +32,767.

3. Binary Arithmetic using 2's Complement & Overflow Detection

Carry discard rule aur Arithmetic Overflow kab hota hai?

2's Complement Addition: Dono numbers ko standard binary addition ki tarah add karo. Agar MSB par carry generate hota hai, to us carry ko simply DISCARD (drop) kar dete hain!

Overflow Condition: Jab do same-sign numbers ko add karne par result opposite sign ka aa jaye, to OVERFLOW occur hota hai (result n-bit capacity se bahar nikal gaya).

Formula: Overflow = C_{in} XOR C_{out} (MSB par aane wala carry aur MSB se bahar nikalne wala carry agar alag-alag hain, to overflow hua hai).

[SOLVED EXAMPLE] CUET PG PYQ: Addition of two 8-bit 2's Complement Numbers

Problem: Given two 8-bit 2's complement numbers A = 11111111 and B = 11111111. Find the sum A + B.

Step-by-Step Solution:

Step 1: Method 1 (Decimal value analysis):

Step 2: A = 11111111: MSB is 1, so negative. Magnitude = 2's complement of 11111111 = 00000000 + 1 = 00000001 (1). So A = -1.

Step 3: B = 11111111: Similarly, B = -1.

Step 4: Sum = A + B = (-1) + (-1) = -2.

Step 5: Now convert -2 into 8-bit 2's complement:

Step 6: +2 in 8-bit binary = 00000010.

Step 7: 1's complement = 11111101. Add 1 -> 11111110.

Step 8: Method 2 (Direct Binary Addition):

Step 9: 11111111
+ 11111111
= (1) 11111110 (Carry out = 1 is discarded in 8-bit system).

Step 10: Result = 11111110.

Final Result: Sum = 11111110 (which equals -2 in decimal).

UNIT / LECTURE 04

Floating Point Representation (IEEE 754 Standard)

Topic Overview: Real number representation, Scientific notation, IEEE 754 32-bit Single Precision and 64-bit Double Precision, Biased exponent, Mantissa normalization, aur Special values (+inf, -inf, NaN).

1. Floating Point Concept & General Form

Scientific notation in binary: Mantissa aur Exponent

Fixed point representation mein fractional range bohot limited hoti hai. Real numbers (bohot chhote ya bohot bade numbers jaise 0.00000012 ya 6.023 * 10^23) ko represent karne ke liye Floating Point Representation use hota hai.

General Binary Scientific Form: N = (-1)^S * (1.M) * 2^E

S = Sign Bit (0 for Positive, 1 for Negative).

M = Mantissa / Fraction (Normalized form mein radix point ke left mein hamesha implicit '1' hota hai, isliye sirf point ke baad ka fraction 'M' store karte hain).

E = True Exponent (Isme bias add karke Biased Exponent store kiya jata hai taaki negative exponents bhi unsigned integer ki tarah store ho sakein).

2. IEEE 754 Formats (Single & Double Precision)

32-bit vs 64-bit field distribution and Bias values

Field	Single Precision (32-bit)	Double Precision (64-bit)
Total Bits	32 bits	64 bits
Sign Bit (S)	1 bit (Bit 31)	1 bit (Bit 63)
Biased Exponent (E)	8 bits (Bits 30 to 23)	11 bits (Bits 62 to 52)
Mantissa / Fraction (F)	23 bits (Bits 22 to 0)	52 bits (Bits 51 to 0)
Exponent Bias Value	127 (2^(8-1) - 1)	1023 (2^(11-1) - 1)
Stored Exponent Range	1 to 254 (0 and 255 reserved)	1 to 2046 (0 and 2047 reserved)

[MUST REMEMBER FOR CUET PG] Special Reserved IEEE 754 Patterns

* E = All 0s, F = All 0s: Represents Zero (S=0 -> +0, S=1 -> -0).
* E = All 0s, F != 0: Denormalized / Subnormal number.
* E = All 1s, F = All 0s: Represents INFINITY (+inf if S=0, -inf if S=1).
* E = All 1s, F != 0: Represents NaN (Not a Number, e.g. 0/0 or sqrt(-1)).

[SOLVED EXAMPLE] CUET PG PYQ: Representation of +Infinity in a Custom 10-bit Float

Problem: Suppose a 10-bit computer uses IEEE float format with 1 sign bit, 5 exponent bits, and 4 fraction bits. What is the representation for +infinity (+inf)?

Step-by-Step Solution:

Step 1: 1. Sign bit (S): Number is positive (+inf), so S = 0.

Step 2: 2. Exponent bits (E): According to IEEE 754 rule, Infinity requires Exponent to be ALL 1s. Exponent has 5 bits, so E = 11111.

Step 3: 3. Fraction bits (F): According to IEEE 754 rule, Infinity requires Fraction to be ALL 0s. Fraction has 4 bits, so F = 0000.

Step 4: Combine fields: [S] [E] [F] = [0] [11111] [0000].

Final Result: 0 11111 0000 (Option matches CUET PG Exam Paper!).

UNIT / LECTURE 05

Computer Arithmetic & Digital Codes

Topic Overview: Binary arithmetic operations, Booth's Algorithm for signed multiplication, BCD arithmetic, Excess-3, Gray Code (Reflected Binary Code) and Weighted vs Non-Weighted codes.

1. Binary Arithmetic & Multiplication

Addition, Subtraction, Carry, Borrow, aur Booth's Algorithm

Binary Addition Rules: 0+0=0, 0+1=1, 1+0=1, 1+1=0 (with Carry 1), 1+1+1=1 (with Carry 1).

Binary Subtraction Rules: 0-0=0, 1-0=1, 1-1=0, 0-1=1 (with Borrow 1 from next higher column).

Booth's Multiplication Algorithm: Signed 2's complement binary numbers ko multiply karne ke liye Andrew Donald Booth ka algorithm standard CPU hardware standard hai. Ye traditional shift-and-add se bohot fast hai kyunki ye consecutive 1s ke block ko skip karta hai.

Booth's Inspection Rule on Multiplier bit pair (Q_i, Q_{i-1}):

- If 0 0: Do nothing, just Arithmetic Shift Right (ASR).
- If 1 1: Do nothing, just Arithmetic Shift Right (ASR).
- If 0 1 (Beginning of a block of 1s): Add Multiplicand M to Accumulator (A = A + M), then ASR.
- If 1 0 (End of a block of 1s): Subtract Multiplicand M from Accumulator (A = A - M), then ASR.

[MUST REMEMBER FOR CUET PG] CUET PG Key Fact: Booth's Algorithm

* Booth's algorithm works on SIGNED 2's complement numbers directly, without needing separate sign handling!
* Best case: Strings of consecutive 0s or 1s (minimum additions/subtractions).
* Worst case: Alternating 010101... (every bit requires addition or subtraction).

2. Digital Codes: BCD, Excess-3, and Gray Code

Weighted vs Non-Weighted codes, Self-complementing codes, aur Unit-distance code

A. BCD (Binary Coded Decimal / 8421 Code): Har decimal digit (0 to 9) ko independently 4-bit binary mein likha jata hai. Valid BCD range = 0000 to 1001 (0 to 9). 1010 se 1111 tak (10 to 15) INVALID BCD states hoti hain! Agar addition mein result > 9 ya carry out aaye, to correct karne ke liye 0110 (6) add kiya jata hai.

B. Excess-3 Code (XS-3): Non-weighted code jisme har decimal digit ke BCD mein 3 (0011) add kar diya jata hai. Example: 4 in Excess-3 = 4 + 3 = 7 = 0111. Property: Excess-3 is a SELF-COMPLEMENTING CODE (1's complement lene par decimal 9's complement mil jata hai)!

C. Gray Code (Reflected Binary Code / Cyclic Code): Gray code ek NON-WEIGHTED and UNIT-DISTANCE CODE hai. Kisi bhi do consecutive numbers ke beech sirf aur sirf EK HI BIT change hoti hai! Is property ki wajah se switches, optical shaft encoders, aur K-Maps mein error bilkul eliminate ho jata hai.

Decimal	Binary (8421)	BCD (8421)	Excess-3 (BCD + 3)	Gray Code
0	0000	0000	0011	0000
1	0001	0001	0100	0001
2	0010	0010	0101	0011
3	0011	0011	0110	0010

Decimal	Binary (8421)	BCD (8421)	Excess-3 (BCD + 3)	Gray Code
4	0100	0100	0111	0110
5	0101	0101	1000	0111
6	0110	0110	1001	0101
7	0111	0111	1010	0100
8	1000	1000	1011	1100
9	1001	1001	1100	1101

[SOLVED EXAMPLE] Binary to Gray Code & Gray to Binary Conversion

Problem: Convert Binary 1011 to Gray Code, and convert Gray Code 1110 back to Binary.

Step-by-Step Solution:

Step 1: Part 1: Binary (B3 B2 B1 B0 = 1 0 1 1) to Gray (G3 G2 G1 G0):

Step 2: Rule: G3 = B3 = 1 (MSB remains same).

Step 3: G2 = B3 XOR B2 = 1 XOR 0 = 1.

Step 4: G1 = B2 XOR B1 = 0 XOR 1 = 1.

Step 5: G0 = B1 XOR B0 = 1 XOR 1 = 0.

Step 6: Resulting Gray code = 1110.

Step 7: Part 2: Gray (G3 G2 G1 G0 = 1 1 1 0) to Binary (B3 B2 B1 B0):

Step 8: Rule: B3 = G3 = 1.

Step 9: B2 = B3 XOR G2 = 1 XOR 1 = 0.

Step 10: B1 = B2 XOR G1 = 0 XOR 1 = 1.

Step 11: B0 = B1 XOR G0 = 1 XOR 0 = 1.

Step 12: Resulting Binary = 1011.

Final Result: Binary 1011 <-> Gray 1110 (Verified both ways).

UNIT / LECTURE 06

Digital Computers Architecture & Organization

Topic Overview: Basic computer organization, CPU registers (PC, IR, MAR, MDR, AC), System Buses, Control Unit (Hardwired vs Microprogrammed), aur Instruction Cycle (Fetch, Decode, Execute).

1. Basic Functional Units & CPU Internal Registers

Central Processing Unit ke components aur special purpose registers

Digital Computer ke teen primary subsystems hote hain: CPU (Processor), Memory Subsystem (RAM/ROM), aur Input/Output (I/O) Subsystem.

CPU ke internal major components hote hain: Arithmetic Logic Unit (ALU), Control Unit (CU), aur Internal Registers.

Important CPU Registers tested in CUET PG:

1. Program Counter (PC): Agle execute hone wale instruction ka memory address hold karta hai. Instruction fetch hote hi PC automatically increment ho jata hai.

2. Instruction Register (IR): Current instruction jo abhi execute ho raha hai, uske opcode aur operands ko store karta hai.

3. Memory Address Register (MAR): Memory ke us specific address ko hold karta hai jahan se data read ya write hona hai. MAR directly Address Bus se connected hota hai.

4. Memory Data Register (MDR) / Memory Buffer Register (MBR): Memory se read kiya gaya data ya memory mein write kiya jane wala data hold karta hai. MDR directly Data Bus se connected hota hai.

5. Accumulator (AC): ALU operations ke intermediate results ko temporarily hold karta hai.

[MUST REMEMBER FOR CUET PG] CUET PG PYQ Match List Concept

* Program Counter (PC) -> Holds address of NEXT instruction to be executed.
* Instruction Register (IR) -> Holds instruction CURRENTLY being executed.
* Memory Data Register (MDR) -> Acts as buffer between CPU and Memory/IO for data transfer.
* Clock -> Generates timing/control pulses for synchronizing execution.

2. Control Unit & Instruction Execution Cycle

Hardwired vs Microprogrammed CU, Fetch-Decode-Execute cycle

Control Unit (CU) pure computer ka supervisor hota hai jo memory, ALU, aur registers ko timing aur control signals bhejta hai.

Do types ke Control Units hote hain:

- Hardwired CU: Combinational logic gates, flip-flops, aur decoders se bana hota hai. Super fast speed hoti hai, lekin modifications impossible/bohot mushkil hote hain (RISC processors mein use hota hai).
- Microprogrammed CU: Control memory (ROM) mein stored microinstructions (control words) se operate hota hai. Slower hota hai par modify karna bohot flexible hota hai (CISC processors mein use hota hai).

Instruction Cycle 3 major phases mein complete hota hai:

Phase 1: Fetch -> PC se address MAR mein jata hai, memory se instruction nikal kar MDR ke through IR mein load hota hai. PC = PC + 1.

Phase 2: Decode -> Control unit IR ke opcode ko decode karta hai aur required operations identify karta hai.

Phase 3: Execute -> ALU aur internal registers operation perform karte hain aur result save karte hain.

UNIT / LECTURE 07

Boolean Algebra Axioms, Laws & Simplification

Topic Overview: Huntington Postulates, Idempotent, Involution, Distributive, Absorption laws, De Morgan's Theorems, Consensus Theorem, Duality Principle aur step-by-step Boolean simplification.

1. Boolean Axioms & Fundamental Laws

Boolean variables (0, 1) aur core simplification identities

Boolean Algebra binary variables aur logical operations ka mathematical framework hai, jise George Boole ne invent kiya tha.

A. Identity Laws: A + 0 = A, A * 1 = A.

B. Null / Domination Laws: A + 1 = 1, A * 0 = 0.

C. Idempotent Laws: A + A = A, A * A = A.

D. Complement Laws: A + A' = 1, A * A' = 0.

E. Involution (Double Negation) Law: (A')' = A.

F. Commutative Laws: A + B = B + A, A * B = B * A.

G. Associative Laws: (A + B) + C = A + (B + C), (A * B) * C = A * (B * C).

H. Distributive Laws (Very Important!):

- Rule 1: A * (B + C) = (A * B) + (A * C)
- Rule 2 (Dual Distributive - Very frequently tested!): A + (B * C) = (A + B) * (A + C)

I. Absorption Laws:

- A + (A * B) = A
- A * (A + B) = A
- A + A'B = A + B (Frequently used in exams!)
- A * (A' + B) = A * B

[IMPORTANT FORMULA / LAW] The Consensus Theorem (Redundancy Rule)

* Standard SOP Form: AB + A'C + BC = AB + A'C
Notice: BC term redundant ho jata hai aur eliminate ho jata hai!
Identification Trick: 3 variables hain (A, B, C). Ek variable 'A' ek term mein uncomplemented (A) aur dusre term mein complemented (A') appear hota hai. Teesra term (BC) un dono ke bache hue variables ka product hota hai. Wahi redundant term eliminate ho jata hai!
* POS Form: (A + B)(A' + C)(B + C) = (A + B)(A' + C).

2. De Morgan's Theorems & Principle of Duality

Complementation of expressions and Dual expressions

De Morgan's First Theorem: (A * B)' = A' + B'
(The complement of a product is equal to the sum of the complements).

De Morgan's Second Theorem: (A + B)' = A' * B'
(The complement of a sum is equal to the product of the complements).

Mnemonic Trick: "Break the line, change the sign!" (Agar upar ka bar todte hain, to AND operation OR ban jata hai, aur OR operation AND ban jata hai).

Principle of Duality: Kisi bhi valid Boolean equation ka DUAL obtain karne ke liye:
1. Saare AND (*) ko OR (+) se replace karo.
2. Saare OR (+) ko AND (*) se replace karo.
3. Saare 1 ko 0 se aur 0 ko 1 se replace karo.
4. Variables ko COMPLEMENT NAHI KARTE (variables same rehte hain)!

[SOLVED EXAMPLE] CUET PG Boolean Simplification Question

Problem: Simplify the Boolean expression: Y = AB + A(B + C) + B(B + C).

Step-by-Step Solution:

Step 1: Given: Y = AB + A(B + C) + B(B + C)

Step 2: Step 1: Expand brackets using distributive law:

Step 3: Y = AB + AB + AC + BB + BC

Step 4: Step 2: Apply Idempotent law (AB + AB = AB, and BB = B):

Step 5: Y = AB + AC + B + BC

Step 6: Step 3: In terms (B + BC), apply Absorption law B + BC = B:

Step 7: Y = AB + AC + B

Step 8: Step 4: In terms (AB + B), apply Absorption law AB + B = B:

Step 9: Y = B + AC

Final Result: Simplified Expression Y = B + AC

UNIT / LECTURE 08

Logic Gates & Universal Logic Realization

Topic Overview: Basic Gates (AND, OR, NOT), Universal Gates (NAND, NOR), Special Gates (XOR, XNOR), Truth Tables, Schematics, Universal Gate Minimal Count Table, aur Positive vs Negative Logic.

1. Basic & Universal Logic Gates

Symbols, Boolean functions, aur working principles

Logic Gates digital systems ke basic physical electronic building blocks hote hain jo binary inputs lekar single binary output produce karte hain.

A. NOT Gate (Inverter): Y = A'. Single input, single output. Input 0 par 1, 1 par 0 deta hai.

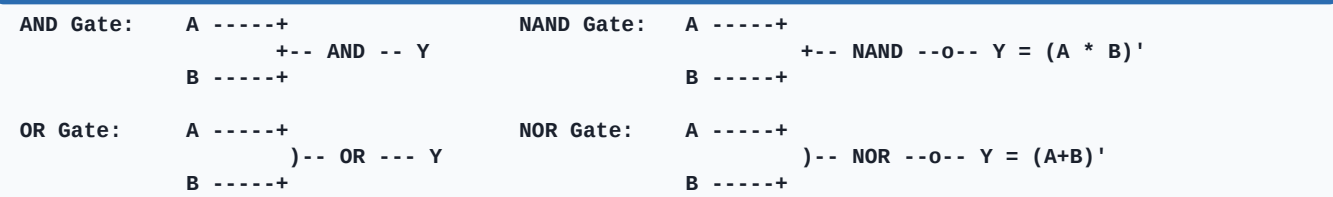
B. AND Gate: Y = A * B. Output 1 tabhi hota hai jab SAARE inputs 1 hon. Kisi bhi input par 0 aane par output 0 ho jata hai.

C. OR Gate: Y = A + B. Output 1 hota hai jab KAM SE KAM EK input 1 ho. Output 0 tabhi hota hai jab saare inputs 0 hon.

D. NAND Gate (Universal Gate): Y = (A * B)'. AND gate ke aage invert lagaya gaya hai. Output tabhi 0 hota hai jab saare inputs 1 hon. Kisi bhi input par 0 aane par output 1 ho jata hai!

E. NOR Gate (Universal Gate): Y = (A + B)'. OR gate ke aage invert lagaya gaya hai. Output tabhi 1 hota hai jab saare inputs 0 hon!

[CIRCUIT SCHEMATIC] Basic & Universal Gates Schematics



Working Principle: NAND aur NOR ko Universal Gates isliye kaha jata hai kyunki sirf aur sirf NAND (ya sirf NOR) gates ka use karke DUNIYA KA KOI BHI boolean function ya circuit banaya ja sakta hai!

2. Exclusive Gates: XOR and XNOR Gates

Odd-function vs Even-function properties and parity detection

A. XOR Gate (Exclusive-OR / Inequality Detector / Odd Function):

Equation: Y = A XOR B = A'B + AB'

Property: Output 1 tabhi hota hai jab dono inputs DIFFERENT hon (0 1 ya 1 0). Multiple inputs ke case mein XOR gate ka output 1 hota hai jab total number of 1s ODD hote hain! Isliye ise Odd 1s Detector kehte hain.

B. XNOR Gate (Exclusive-NOR / Equivalence Gate / Even Function):

Equation: Y = (A XOR B)' = AB + A'B' = A XNOR B

Property: Output 1 tabhi hota hai jab dono inputs IDENTICAL (same) hon (0 0 ya 1 1). Multiple inputs mein XNOR output 1 deta hai jab number of 1s EVEN hote hain!

A	B	AND (A * B)	OR (A+B)	NAND ((A * B)')	NOR ((A+B)')	XOR (A XOR B)	XNOR (A XNOR B)
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

[MUST REMEMBER FOR CUET PG] CUET PG Super-Repeated Table: Minimum Universal Gates Required

Har exam mein pucha jata hai ki kisi gate ko implement karne ke liye kitne minimum NAND ya NOR gates lagte hain:

Target Function / Gate | Min NAND Gates Required | Min NOR Gates Required

---

NOT Gate | 1 | 1
AND Gate | 2 | 3
OR Gate | 3 | 2
NOR Gate | 4 | 1
NAND Gate | 1 | 4
XOR Gate | 4 | 5
XNOR Gate | 5 | 4

Mnemonic Trick: XOR banane ke liye 4 NAND lagte hain, XNOR ke liye 5 NAND lagte hain. NOR gate ke case mein ye ulta (flip) ho jata hai: XOR ke liye 5 NOR aur XNOR ke liye 4 NOR!

UNIT / LECTURE 09

Standard & Canonical Forms (SOP & POS)

Topic Overview: Minterms (m), Maxterms (M), Canonical Sum of Products (SOP / Sigma), Canonical Product of Sums (POS / Pi), Form conversion techniques, aur Truth Table representation.

1. Minterms and Canonical SOP Form

Sum of Products kya hai aur truth table se kaise generate karte hain?

Minterm (Product Term): Ek aisa Boolean product term jisme function ke SAARE variables exactly ek baar aate hain (chahe normal form mein hon ya complemented form mein).

SOP Convention: Variable ki value 1 hone par use uncomplemented (A) likhte hain, aur variable ki value 0 hone par use complemented (A') likhte hain.

Minterm Notation: 3 variables A, B, C ke liye 2^3 = 8 possible minterms hote hain (m_0 se m_7):

- m_0 = A'B'C' (000), m_1 = A'B'C (001), m_2 = A'BC' (010), m_3 = A'BC (011)
- m_4 = AB'C' (100), m_5 = AB'C (101), m_6 = ABC' (110), m_7 = ABC (111)

Canonical SOP (Sum of Minterms): Truth table mein jahan jahan output F = 1 hota hai, un saare corresponding minterms ka logical OR (+) kar dete hain.

SOP Sigma Notation: F(A, B, C) = Sigma m(1, 4, 7) = m_1 + m_4 + m_7.

2. Maxterms and Canonical POS Form

Product of Sums kya hai aur Minterm-Maxterm relationship kya hai?

Maxterm (Sum Term): Ek aisa Boolean sum term jisme function ke SAARE variables exactly ek baar aate hain.

POS Convention (ULTO TRICK!): Variable ki value 0 hone par use normal (A) likhte hain, aur variable ki value 1 hone par use complemented (A') likhte hain!

Maxterm Notation (M_0 to M_7):

- M_0 = (A + B + C) (000), M_1 = (A + B + C') (001), M_2 = (A + B' + C) (010)...

Canonical POS (Product of Maxterms): Truth table mein jahan jahan output F = 0 hota hai, un saare corresponding maxterms ka logical AND (*) kar dete hain.

POS Pi Notation: F(A, B, C) = Pi M(0, 2, 3, 5, 6).

[MUST REMEMBER FOR CUET PG] Direct Conversion between Sigma m and Pi M

* Relationship: Total possible terms for n variables = 2^n.
* Agar kisi function ka SOP form hai F = Sigma m(1, 2, 5, 7), to uska POS form wahi bache hue indices honge: F = Pi M(0, 3, 4, 6)!
* Complement Function: F' = Sigma m(0, 3, 4, 6) = Pi M(1, 2, 5, 7).
* Minterm aur Maxterm ek dusre ke complement hote hain: M_i = (m_i)'.

[SOLVED EXAMPLE] Truth Table to Boolean Canonical SOP Form

Problem: Find the canonical SOP expression for a 3-variable function f(x, y, z) that produces output 1 for input combinations: (0, 1, 0), (1, 0, 1), and (1, 1, 1).

Step-by-Step Solution:

Step 1: Combination 1: x = 0, y = 1, z = 0 (Decimal 2): Minterm m_2 = x' y z'.

Step 2: Combination 2: x = 1, y = 0, z = 1 (Decimal 5): Minterm m_5 = x y' z.

Step 3: Combination 3: x = 1, y = 1, z = 1 (Decimal 7): Minterm m_7 = x y z.

Step 4: Combine all minterms using OR operation:

Step 5: f(x, y, z) = m_2 + m_5 + m_7 = x' y z' + x y' z + x y z.

Step 6: In Sigma notation: f(x, y, z) = Sigma m(2, 5, 7).

Final Result: f(x, y, z) = x' y z' + x y' z + x y z (Sigma m(2, 5, 7)).

UNIT / LECTURE 10

Karnaugh Maps (K-Maps) Part 1: Fundamentals & 2, 3-Variables

Topic Overview: Purpose of K-Maps, Gray code coordinate ordering, Cell numbering, 2-variable and 3-variable map grouping, Prime Implicants (PI) aur Essential Prime Implicants (EPI).

1. K-Map Architecture & Gray Code Ordering

Maurice Karnaugh ka graphical minimization tool

Karnaugh Map (K-Map) Boolean expressions ko visually minimize karne ka graphical method hai jo algebraic laws apply karne mein hone wali human errors ko prevent karta hai.

Cells Count: Ek n-variable Boolean function ke liye K-Map mein exactly 2^n cells hoti hain. (2 variables -> 4 cells, 3 variables -> 8 cells, 4 variables -> 16 cells).

Gray Code Ordering Rule: K-Map ke rows aur columns ko binary counting (00, 01, 10, 11) se label NAHI karte! Unhe GRAY CODE (00, 01, 11, 10) se label kiya jata hai!

Kyu Gray Code? Taaki kisi bhi do adjacent (pass-pass wali) cells ke coordinate mein sirf aur sirf EK HI BIT ka difference ho, jisse common variable factor out ho sake!

[EXAM TRAP / COMMON CONFUSION] 3-Variable Cell Numbering Trap

Notice the column sequence in 3-variable K-Map (A on row, BC on columns):
Columns are: 00 (0), 01 (1), 11 (3), 10 (2)!
Row 0 (A=0): Cells are 0, 1, 3, 2 (Notice 3 comes before 2!)
Row 1 (A=1): Cells are 4, 5, 7, 6 (Notice 7 comes before 6!)

2. Grouping Rules, Prime Implicants & Essential Prime Implicants

Pairs, Quads, Octets, aur Redundant group identification

Grouping Rules:

1. Group size hamesha 2 ki power mein hona chahiye: 1, 2 (Pair), 4 (Quad), 8 (Octet), 16.

2. Diagonal grouping bilkul ALLOWED NAHI HAI! Group sirf horizontal ya vertical rectangular blocks mein ban sakta hai.

3. Har group jitna BADA ban sake utna bada banana chahiye (Bada group = Kam variables in final term!).

4. Groups ek dusre ko OVERLAP kar sakte hain.

5. K-Map edges wrap around hoti hain: extreme left column extreme right column se adjacent hota hai!

Prime Implicant (PI): K-Map mein banne wala aisa sabse bada possible group jise aur kisi bade group mein merge na kiya ja sake.

Essential Prime Implicant (EPI): Aisa Prime Implicant jisme kam se kam EK '1' aisa ho jo kisi aur group mein cover NAHI ho raha ho (akela ho)!

Group Name	Number of Cells (2^k)	Variables Eliminated	Term Size (in 4-variable map)
Single Cell	1 cell (2^0)	0 variables eliminated	Full 4-variable term (e.g. ABCD)
Pair	2 cells (2^1)	1 variable eliminated	3-variable term (e.g. ABC)
Quad	4 cells (2^2)	2 variables eliminated	2-variable term (e.g. AB)
Octet	8 cells (2^3)	3 variables eliminated	1-variable term (e.g. A)

Group Name	Number of Cells (2^k)	Variables Eliminated	Term Size (in 4-variable map)
Hexadecet	16 cells (2^4)	4 variables eliminated	Constant 1 (Independent of inputs)

UNIT / LECTURE 11

Karnaugh Maps (K-Maps) Part 2: 4-Variables & Don't Care Conditions

Topic Overview: 4-variable K-map layout, 4-corner wrap-around grouping, Don't-Care conditions (X / d), Product of Sums (POS) K-map minimization, aur Minimal expression derivations.

1. 4-Variable K-Map (16 Cells) & Corner Grouping

AB rows and CD columns layout aur toroidal wrap-around

4-variable K-Map mein 16 cells hoti hain (Rows: AB = 00, 01, 11, 10; Columns: CD = 00, 01, 11, 10).

Cell Numbers Layout:

Row 00 (AB=00): 0 1 3 2

Row 01 (AB=01): 4 5 7 6

Row 11 (AB=11): 12 13 15 14 (Notice Row 11 comes before 10!)

Row 10 (AB=10): 8 9 11 10

Special 4-Corner Quad: Cells (0, 2, 8, 10) aapas mein ek valid Quad banati hain! Kyunki left-right wrap-around aur top-bottom wrap-around dono exist karte hain. Term formed = B' D'.

[CIRCUIT SCHEMATIC] 4-Variable K-Map Grid Matrix

```
  CD ->   00      01      11      10
AB = 00 [  0 ]  [  1 ]  [  3 ]  [  2 ]
AB = 01 [  4 ]  [  5 ]  [  7 ]  [  6 ]
AB = 11 [ 12 ]  [ 13 ]  [ 15 ]  [ 14 ]
AB = 10 [  8 ]  [  9 ]  [ 11 ]  [ 10 ]
```

Working Principle: Cells 0, 2, 8, 10 are adjacent corners. Edge columns 00 and 10 are adjacent. Edge rows 00 and 10 are adjacent.

2. Don't-Care Conditions (X / d)

Don't-care conditions ko kab include karna chahiye aur kab ignore?

Digital design mein kuch aisi input combinations hoti hain jo system mein kabhi occur hi nahi hoti (jaise BCD mein 1010 se 1111 tak ke 6 combinations invalid hote hain), ya unka output don't matter hota hai. Aise terms ko Don't-Care terms kehte hain, jinhe 'X' ya 'd' se represent karte hain.

Golden Rules for Don't-Care Conditions in K-Map:

1. Don't care 'X' ko '1' treat karke group ko BADA banane ke liye use kiya ja sakta hai.

2. Agar kisi 'X' se group bada nahi ban raha, to use zabardasti group mein include karne ki BILKUL zaroorat nahi hai (use 0 treat karke chhod do)!

3. Sirf 'X' ka alag se koi independent group kabhi nahi banaya jata!

[SOLVED EXAMPLE] CUET PG DPP Question: K-Map Simplification with Don't Cares

Problem: Minimize the Boolean function: F(A, B, C, D) = Sigma m(1, 3, 7, 11, 15) + d(0, 2, 5).

Step-by-Step Solution:

Step 1: Plot 1s at minterms: 1, 3, 7, 11, 15.

Step 2: Plot Xs at don't cares: 0, 2, 5.

Step 3: Step 1: Check minterm 15, 11, 7, 3: Column CD = 11 mein cells (3, 7, 15, 11) hain. Ye charo 1 hain, to pura column quad banata hai: Cells (3, 7, 11, 15) -> Term = CD.

Step 4: Step 2: Remaining minterm is 1. Check if it can form a quad using don't cares (0, 2, 5) and minterm 3.

Step 5: Row 00 has cells (0, 1, 3, 2). Cell 1 and 3 are 1s, and cells 0 and 2 are don't cares (X).

Step 6: Taking cells (0, 1, 2, 3) gives a complete Quad of row 00! -> Term = A'B'.

Step 7: All minterms are now covered.

Step 8: Combine essential terms: F = CD + A'B'.

Final Result: F = CD + A'B' (Simplified minimal SOP form).

UNIT / LECTURE 12

Combinational Logic Circuits: Adders & Subtractors

Topic Overview: Combinational vs Sequential definition, Half Adder, Full Adder, Half Subtractor, Full Subtractor, Parallel Binary Adder, Carry Lookahead Adder, and Logic Equations.

1. Combinational Circuits Overview

Combinational circuit ki core properties aur timing characteristics

Combinational Circuit ek aisa digital circuit hota hai jiska output kisi bhi instant par SIRF AUR SIRF PRESENT INPUTS par depend karta hai. Isme koi internal memory element nahi hota, na hi output se input ki taraf koi feedback loop hota hai!

Examples: Adders, Subtractors, Multiplexers, Demultiplexers, Encoders, Decoders, Code Converters, Comparators.

Speed: Memory elements aur clock na hone ke karan combinational circuits sequential circuits ki tulna mein significantly faster hote hain (propagation delay is determined solely by logic gates).

2. Half Adder and Full Adder Circuits

Schematics, Truth Tables, Sum and Carry equations

A. Half Adder: Do single-bit binary inputs (A aur B) ko add karta hai aur 2 outputs produce karta hai: Sum (S) aur Carry (C). Ye previous stage ka carry input accept NAHI kar sakta (isliye ise 'Half' Adder kehte hain).

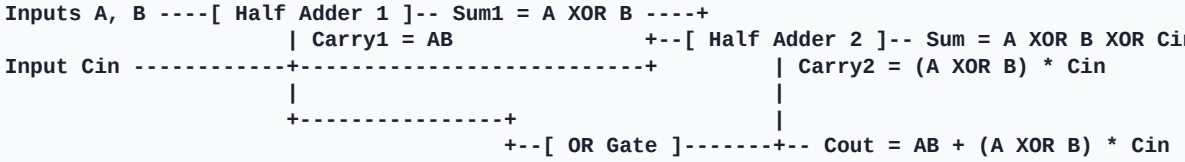
- Sum equation: S = A XOR B = A'B + AB'
- Carry equation: C = A * B
- Minimum gates: 1 XOR gate aur 1 AND gate (Total 2 gates). Minimum universal NAND gates required = 5 NAND gates.

B. Full Adder: Teen single-bit binary inputs (A, B, aur Carry input C_in) ko add karta hai aur 2 outputs deta hai: Sum (S) aur Carry out (C_out).

- Sum equation: S = A XOR B XOR C_in
- Carry out equation: C_out = AB + BC_in + AC_in = AB + C_in(A XOR B)
- Realization using Half Adders: Ek Full Adder ko TWO HALF ADDERS aur ONE OR GATE ka use karke construct kiya ja sakta hai!

A	B	Cin	Sum (S = A XOR B XOR Cin)	Cout (AB + Cin(A XOR B))
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

[CIRCUIT SCHEMATIC] Full Adder using Two Half Adders and One OR Gate



Working Principle: Standard CUET PG Question: How many Half Adders and OR gates are required to build a Full Adder? Answer: 2 Half Adders and 1 OR Gate.

3. Half Subtractor and Full Subtractor Circuits

Difference and Borrow equations and realization

A. Half Subtractor: Do single-bit binary numbers A (minuend) aur B (subtrahend) ko subtract karta hai (A - B):

- Difference (D): D = A XOR B
- Borrow (B_out): B_out = A' * B (Notice A is complemented!)

B. Full Subtractor: Teen inputs A, B, aur Borrow-in B_in ko subtract karta hai (A - B - B_in):

- Difference (D): D = A XOR B XOR B_in
- Borrow out: B_out = A'B + A'B_in + BB_in = A'B + B_in(A XOR B)'
- Realization: Ek Full Subtractor banane ke liye 2 Half Subtractors aur 1 OR Gate lagta hai!

[MUST REMEMBER FOR CUET PG] Comparison: Adder vs Subtractor equations

* Full Adder Sum = A XOR B XOR Cin | Full Subtractor Diff = A XOR B XOR Bin (EXACTLY SAME!)
* Full Adder Carry = AB + Cin(A XOR B) | Full Subtractor Borrow = A'B + Bin(A XOR B)' (Notice the complement on A and on XOR term!).

UNIT / LECTURE 13

Multiplexers, Demultiplexers, Decoders & Encoders

Topic Overview: Data Selectors (MUX), Data Distributors (DEMUX), n-to-2^n Decoders, 2^n-to-n Encoders, Priority Encoders, aur MUX-based Boolean logic implementation.

1. Multiplexer (MUX) - Data Selector

Multiple inputs se single output select karna

Multiplexer (MUX) ek aisa combinational circuit hai jo kai saare input lines (2^n inputs) mein se kisi ek input line ko select karke single output line par forward karta hai.

Selection mechanism: MUX ke paas n SELECT LINES (Control lines) hoti hain. Agar MUX mein 2^n inputs hain, to exactly n select lines lagengi.

Formula: Total Inputs = 2^n <==> Select Lines = n

- 2-to-1 MUX: 2 inputs (I0, I1), 1 select line (S0). Output Y = S0' I0 + S0 I1.
- 4-to-1 MUX: 4 inputs (I0 to I3), 2 select lines (S1, S0). Output Y = S1'S0'I0 + S1'S0 I1 + S1 S0'I2 + S1 S0 I3.
- 8-to-1 MUX: 8 inputs, 3 select lines (2^3 = 8).
- 16-to-1 MUX: 16 inputs, 4 select lines (2^4 = 16).

Property: Multiplexer ko Universal Logic Circuit bhi kaha jata hai kyunki kisi bhi Boolean function ko bina kisi extra gate ke ek single MUX se implement kiya ja sakta hai!

[CIRCUIT SCHEMATIC] 4-to-1 Multiplexer Schematic

```
Input I0 ------+
Input I1 ------+
Input I2 ------+---[ 4-to-1 MUX ]--- Output Y
Input I3 ------+          ||
                    Select Lines (S1, S0)
```

Working Principle: When S1 S0 = 00 -> Y = I0; when S1 S0 = 01 -> Y = I1; when S1 S0 = 10 -> Y = I2; when S1 S0 = 11 -> Y = I3.

[MUST REMEMBER FOR CUET PG] CUET PG Repeated Rule: Select Lines Count

* Question: How many select lines will a 16-to-1 multiplexer have? -> Answer: 4 select lines (2^4 = 16).
* How many 2-to-1 MUX are required to build an 8-to-1 MUX? -> 7 (8/2 + 4/2 + 2/2 = 4 + 2 + 1 = 7 MUX).
* How many 4-to-1 MUX to build 16-to-1 MUX? -> 5 (16/4 = 4, then 4/4 = 1, total 4 + 1 = 5 MUX).

2. Demultiplexer (DEMUX) - Data Distributor

Single input ko multiple outputs mein route karna

Demultiplexer (DEMUX) multiplexer ka theek reverse (opposite) operation perform karta hai. Ye single input data ko n select lines ki setting ke aadhar par 2^n output lines mein se kisi ek line par bhejta hai.

Formula: 1 input, n select lines, 2^n output lines (e.g. 1-to-4 DEMUX, 1-to-8 DEMUX).

Relationship: Ek Decoder jisme Enable (E) pin ko data input bana diya jaye, wo EXACTLY ek Demultiplexer ban jata hai!

3. Decoders and Encoders

Binary code conversion and Priority Encoders

A. Decoder: Ek combinational circuit jo n binary input lines ko decode karke maximum 2^n unique output lines activate karta hai (n-to-2^n Decoder, e.g. 2-to-4 Decoder, 3-to-8 Decoder, 4-to-16 Decoder). Memory addressing aur display driving (BCD to 7-segment) mein use hota hai.

B. Encoder: Decoder ka inverse circuit jo 2^n input lines ko n binary output lines mein compress karta hai (e.g. 8-to-3 Octal-to-Binary encoder, 10-to-4 Decimal-to-BCD encoder).

C. Priority Encoder: Standard encoder mein agar ek se zyada inputs ek sath high (1) ho jayein to ambiguous/galat output aata hai. Priority Encoder mein har input ki priority fixed hoti hai (usually highest index bit has highest priority); agar multiple inputs active hon, to highest priority input ka code hi output par generate hota hai.

Component	Inputs Count	Outputs Count	Select Lines	Primary Application
Multiplexer (MUX)	2^n inputs	1 single output	n select lines	Data selection, Parallel-to-Serial conversion
Demultiplexer (DEMUX)	1 single input	2^n outputs	n select lines	Data distribution, Serial-to-Parallel routing
Decoder	n input lines	2^n output lines	None (Has Enable E)	Memory address decoding, 7-Segment
Encoder	2^n input lines	n output lines	None	Keyboard input encoding, Signal compression

UNIT / LECTURE 14

Sequential Logic Fundamentals & Latches

Topic Overview: Combinational vs Sequential circuits, Synchronous vs Asynchronous, Level vs Edge Triggering, Setup & Hold times, SR Latch using NAND & NOR gates, and Forbidden states.

1. Combinational versus Sequential Circuits

Memory feedback loop aur timing dependency ka difference

Digital circuits do broad categories mein divide hote hain:

1. Combinational Circuits: Output sirf present inputs par depend karta hai. Koi memory element nahi hota, koi clock pulse nahi hoti. (Adders, MUX, Decoders).

2. Sequential Circuits: Output PRESENT INPUTS aur PREVIOUS OUTPUT (Present State) dono par depend karta hai! Isme feedback loop aur memory elements (Latches / Flip-Flops) present hote hain. Clock signal se synchronize hote hain. (Flip-Flops, Registers, Counters).

Feature	Combinational Circuit	Sequential Circuit
Output Depends On	Current inputs only	Current inputs + Previous state (Memory)
Memory Elements	Not required / Absent	Mandatory (Latches / Flip-Flops)
Feedback Path	No feedback path	Feedback path from output to input exists
Clock Signal	Not clock-dependent	Generally clock-dependent (Synchronous)
Operating Speed	Faster (No state waiting)	Comparatively slower (Clock cycle constrained)
Design Complexity	Simpler to design	More complex to design and analyze
Standard Examples	Adder, Subtractor, MUX, Decoder	Flip-Flops, Shift Registers, Counters

2. Latches versus Flip-Flops & Triggering Types

Level-triggered vs Edge-triggered memory elements

A. Latch: Ek LEVEL-TRIGGERED memory storage element hota hai. Jab tak enable signal high (active) rehta hai, input mein hone wala koi bhi change turant output par reflect ho jata hai (Transparent nature). Temporary data latching mein use hota hai.

B. Flip-Flop: Ek EDGE-TRIGGERED memory storage element hota hai. Output sirf aur sirf clock transition ke sharp edge par change hota hai (ya to 0-to-1 Rising Edge par, ya 1-to-0 Falling Edge par). Steady clock level ke dauran input badalne par bhi output change nahi hota!

C. Timing Parameters:

- Setup Time (t_setup): Clock edge aane se theek PEHLE kitni der tak input data constant aur stable rehna chahiye.
- Hold Time (t_hold): Clock edge trigger hone ke theek BAAD kitni der tak input data stable rehna chahiye.

[MUST REMEMBER FOR CUET PG] CUET PG PYQ Match List: Triggering Types

* Positive Edge Triggering: Clock transition from 0 to 1 (Rising edge / Low to High).
* Negative Edge Triggering: Clock transition from 1 to 0 (Falling edge / High to Low).
* Level Triggering: Active continuously throughout the high (1) or low (0) duration of the pulse.

3. SR Latch (NAND Gate & NOR Gate Implementation)

Set-Reset latch behavior aur invalid/forbidden condition

SR Latch basic bistable multivibrator hota hai jo 1 bit store karta hai.

NOR Gate SR Latch (Active-High Inputs):

- S=0, R=0: No Change (Hold / Memory state, Q_{next} = Q_n).
- S=0, R=1: Reset state (Q = 0, Q' = 1).
- S=1, R=0: Set state (Q = 1, Q' = 0).
- S=1, R=1: FORBIDDEN / INVALID STATE! (Dono outputs Q aur Q' ek sath 0 ho jate hain, jo logical contradiction hai!).

NAND Gate SR Latch (Active-Low Inputs S' R'):

- S=1, R=1: No Change (Hold state).
- S=0, R=1: Set state (Q = 1).
- S=1, R=0: Reset state (Q = 0).
- S=0, R=0: FORBIDDEN / INVALID STATE!

UNIT / LECTURE 15

Flip-Flops: SR, JK, D, T & Flip-Flop Conversions

Topic Overview: Clocked Flip-Flops, JK Race-Around Condition, Master-Slave JK Flip-Flop, D Flip-Flop (Data), T Flip-Flop (Toggle), Characteristic equations, Excitation tables, aur Flip-Flop Conversions.

1. Clocked SR and JK Flip-Flops

Eliminating the invalid state and the Race-Around Condition

A. Clocked SR Flip-Flop: SR Latch ke aage clock-gated AND/NAND gates laga diye jate hain taaki state transition clock pulse par hi ho. Restriction: S aur R dono ek sath 1 nahi ho sakte (S * R = 0 rule).

Characteristic Equation of SR Flip-Flop: Q(t+1) = S + R' Q(t) (Valid under condition S * R = 0).

B. JK Flip-Flop (Universal Flip-Flop): SR flip-flop ke invalid state problem ko solve karne ke liye output Q aur Q' ka cross-feedback input gates par diya gaya (Jack Kilby ke naam par JK Flip-Flop).

- J=0, K=0: Hold / No Change (Q(t+1) = Q(t)).
- J=0, K=1: Reset (Q(t+1) = 0).
- J=1, K=0: Set (Q(t+1) = 1).
- J=1, K=1: TOGGLE! (Q(t+1) = Q'(t), current state invert ho jati hai!). Invalid state permanently eliminate ho gayi!

Characteristic Equation of JK Flip-Flop: Q(t+1) = J Q'(t) + K' Q(t).

[EXAM TRAP / COMMON CONFUSION] The Race-Around Condition in JK Flip-Flop

* Problem: Race-around tab occur hota hai jab:
1. Flip-flop LEVEL-TRIGGERED ho.
2. Dono inputs J = 1 aur K = 1 hon.
3. Clock pulse width (t_p) flip-flop ke propagation delay (Delta_t) se BADI ho (t_p > Delta_t)!
Is dauran output 0->1->0->1 continuously toggle karta rehta hai aur clock pulse end hone par final output UNCERTAIN ho jata hai!
* Solutions:
1. Use EDGE-TRIGGERED flip-flops.
2. Use MASTER-SLAVE JK FLIP-FLOP (Most popular solution in CUET PG!).
3. Keep clock pulse duration extremely narrow (t_p < Delta_t).

2. D Flip-Flop and T Flip-Flop

Delay/Data flip-flop aur Toggle frequency divider

A. D Flip-Flop (Data / Delay Flip-Flop): SR flip-flop mein S=D aur R=D' (Inverter lagakar) connect kar diya jata hai. Isme single data input D hota hai.

- Behavior: Next state hamesha input D ko follow karti hai: Q(t+1) = D.
- Advantage: Koi invalid state nahi hoti. Computer registers aur pipeline staging buffers mein standard storage unit hota hai.

B. T Flip-Flop (Toggle Flip-Flop): JK flip-flop ke dono inputs ko short karke single input T bana diya jata hai (J = T, K = T).

- When T = 0: No Change (Q(t+1) = Q(t)).
- When T = 1: Toggle (Q(t+1) = Q'(t)).

- Characteristic Equation: Q(t+1) = T XOR Q(t) = T' Q(t) + T Q'(t).
- Application: Binary Counters aur Frequency Division. Agar T=1 continuously rakhein, to output frequency clock frequency ki aadhi ho jati hai: f_{out} = f_{clk} / 2!

Flip-Flop Type	Inputs	Characteristic Equation Q(t+1)	Primary Function
SR Flip-Flop	S, R	Q(t+1) = S + R' Q(t) [SR=0]	Basic Set/Reset storage
JK Flip-Flop	J, K	Q(t+1) = J Q'(t) + K' Q(t)	No invalid state, toggles when J=K=1
D Flip-Flop	D	Q(t+1) = D	Single-bit data storage and delay
T Flip-Flop	T	Q(t+1) = T XOR Q(t)	Frequency divider and binary counters

3. Excitation Tables & Flip-Flop Conversions

Present state se Next state transition ke liye required inputs

Excitation Table ye batati hai ki agar present state Q(t) se desired next state Q(t+1) par jana hai, to inputs kya hone chahiye:

- SR Excitation Table: 0->0: S=0, R=X; 0->1: S=1, R=0; 1->0: S=0, R=1; 1->1: S=X, R=0.
- JK Excitation Table: 0->0: J=0, K=X; 0->1: J=1, K=X; 1->0: J=X, K=1; 1->1: J=X, K=0.
- D Excitation Table: Q(t+1) = D (Always D = Q(t+1)).
- T Excitation Table: 0->0: T=0; 0->1: T=1; 1->0: T=1; 1->1: T=0 (T = Q(t) XOR Q(t+1)).

Flip-Flop Conversion Formulas:

- JK to T: Connect J = T, K = T.
- JK to D: Connect J = D, K = D'.
- SR to D: Connect S = D, R = D'.
- D to T: Connect D = T XOR Q(t).

UNIT / LECTURE 16

Registers & Counters

Topic Overview: Shift Registers (SISO, SIPO, PISO, PIPO), Universal Shift Register, Ring Counter, Johnson Counter, Asynchronous Ripple Counters, Synchronous Counters, and Modulo-N design.

1. Shift Registers (SISO, SIPO, PISO, PIPO)

Data movement and serial-to-parallel conversions

Register n flip-flops ka group hota hai jo n-bit binary data store karta hai. Shift Register data bits ko clock pulses ke sath left ya right shift kar sakta hai.

Four Configurations of Shift Registers:

1. Serial-In Serial-Out (SISO): Data bit-by-bit serial line se enter hota hai aur bit-by-bit serial line se exit hota hai. Sabse slow speed. N-bit data ko store karne ke liye N clock pulses lagti hain, aur retrieve karne ke liye (N-1) clock pulses lagti hain.

2. Serial-In Parallel-Out (SIPO): Data serially input hota hai, lekin saare flip-flops ke outputs ek sath parallel read kiye jate hain. Serial communication receivers mein use hota hai.

3. Parallel-In Serial-Out (PISO): Data ek sath parallel load hota hai aur clock pulses par serial stream ban kar nikalta hai. Serial transmission transmitters mein use hota hai.

4. Parallel-In Parallel-Out (PIPO): Data ek single clock pulse par parallel load hota hai aur turant parallel read hota hai. Fastest shift register! (1 clock pulse for load, 0 clock pulses for read).

2. Ring Counter and Johnson (Twisted Ring) Counter

Shift register based cyclic counters comparison

A. Ring Counter: Ek basic circular shift register jisme last flip-flop ka normal output Q pehle flip-flop ke input D mein feed back kar diya jata hai. Isme initial state '1000...' (single 1) circulate hoti hai.

- Modulus (Distinct States) = N states (equal to number of flip-flops!).
- Number of unused / invalid states = 2^N - N.

B. Johnson Counter (Twisted Ring Counter / Moebius Counter): Last flip-flop ka INVERTED output Q' pehle flip-flop ke input D mein feed back kiya jata hai.

- Modulus (Distinct States) = 2N states (Double the number of flip-flops!).
- Number of unused states = 2^N - 2N.

[MUST REMEMBER FOR CUET PG] CUET PG High-Yield: Ring vs Johnson Comparison Table

For N = 4 Flip-Flops:
* Ring Counter: MOD = N = 4 states. Valid states = 4, Unused states = 16 - 4 = 12.
* Johnson Counter: MOD = 2N = 8 states. Valid states = 8, Unused states = 16 - 8 = 8.
Exam Question: "How many unused states are there in a 4-bit Johnson counter?" -> Answer: 8 states!

3. Asynchronous (Ripple) vs Synchronous Counters

Clock connection difference, Propagation delay, and Modulo-N design

Counter ek sequential circuit hai jo input clock pulses ko count karta hai.

A. Asynchronous (Ripple) Counter: Saare flip-flops ko same clock pulse NAHI milti! Sirf pehle flip-flop (LSB) ko external clock milti hai, aur har agle flip-flop ki clock previous flip-flop ke output Q se trigger hoti hai.

- Disadvantage: Cumulative propagation delay (t_total = N * t_pd). High frequencies par glitch aur speed limitation hoti hai.

B. Synchronous Counter: SAARE FLIP-FLOPS ko EK SATH SIMULTANEOUSLY common external clock signal milti hai! Saare flip-flops ek sath change hote hain. High-speed counting ke liye ideal hota hai.

Modulo-N Counter Rule: Modulo-N counter N distinct states count karta hai (0 to N-1). Iske liye minimum kitne flip-flops (n) chahiye? Formula: 2^(n-1) < N <= 2^n.

UNIT / LECTURE 17

Memory Unit, Cache Organization & CUET PG PYQs

Topic Overview: Memory Hierarchy, RAM (SRAM vs DRAM), ROM types (PROM, EPROM, EEPROM, Flash), Cache Memory Mapping (Direct, Associative, Set-Associative), Hit/Miss ratio, aur CUET PG Previous Years Patterns.

1. Memory Hierarchy & Characteristics

Speed, Cost, and Capacity trade-off

Memory hierarchy system cost ko minimize karte hue CPU execution speed ko maximize karne ke liye design ki gayi hai.

Hierarchy levels from top (fastest) to bottom (slowest):

1. CPU Internal Registers: Fastest access time (~1 clock cycle), smallest capacity (< 1 KB), highest cost.
2. Cache Memory (L1, L2, L3): Very high speed (SRAM based), located close to CPU, size from 32 KB to 32 MB.
3. Main Memory (RAM): Semiconductor dynamic memory (DRAM based), size 8 GB to 64 GB.
4. Secondary Storage: Magnetic HDD, SSD, NVMe Flash, non-volatile, large capacity (512 GB to 4 TB).
5. Tertiary / Offline Backup Storage: Magnetic Tape, Optical Disc (Blu-ray), archival backup storage.

[MUST REMEMBER FOR CUET PG] CUET PG PYQ Match List: Backup Devices

* Question from Class 17: Which of the following storage devices can be used as a BACK-UP device?
(A) HDD (B) ROM (C) RAM (D) Cache (E) Magnetic Tape
* Correct Answer: Only HDD and Magnetic Tape! (RAM and Cache are volatile; ROM is not used for backing up user files).

2. RAM (SRAM vs DRAM) and ROM Technologies

Static vs Dynamic RAM, Refresh cycles, and Flash Memory

A. SRAM (Static RAM): Bipolar ya CMOS flip-flops (4 to 6 transistors) se bana hota hai. Super fast speed hoti hai. ISME PERIODIC REFRESHING KI KOI ZAROORAT NAHI HOTI! Cache memory banane mein use hota hai. Expensive aur power consuming hota hai.

B. DRAM (Dynamic RAM): Har memory cell ek single transistor aur ek microscopic CAPACITOR se bana hota hai. Capacitor charge leak karta hai, isliye DRAM ko PERIODICALLY REFRESH KARNA MANDATORY HOTA HAI! Slower hota hai lekin dense aur sasta hota hai. Main Memory (RAM) banane mein use hota hai.

C. ROM Types: Mask ROM (factory programmed), PROM (one-time programmable by user), EPROM (erased using Ultraviolet UV light), EEPROM (electrically erasable and reprogrammable byte-by-byte), Flash Memory (block-erasable non-volatile memory, used in SSDs and USB pen drives).

Feature	SRAM (Static RAM)	DRAM (Dynamic RAM)
Storage Element	Flip-flop latch (Transistors)	Capacitor + Single Transistor
Refreshing Required?	NO refreshing required	YES, mandatory periodic refresh cycles
Operating Speed	Very Fast (2 to 10 ns)	Slower (30 to 70 ns)
Density & Capacity	Low density (Bulkier cells)	High density (Very compact cells)
Power Consumption	Lower dynamic power	Higher due to continuous refresh

Feature	SRAM (Static RAM)	DRAM (Dynamic RAM)
Primary Application	CPU Cache memory (L1, L2, L3)	Main System Memory (RAM modules)

3. Cache Memory Organization & Mapping Schemes

Direct, Fully Associative, Set-Associative Mapping, and Hit Ratio

Cache Memory CPU aur Main Memory ke beech intermediary buffer ka kaam karta hai. Ye Principle of Locality of Reference (Temporal Locality: recently accessed data access hone ki probability high hai; Spatial Locality: adjacent memory locations access hone ki probability high hai) par kaam karta hai.

Cache Mapping Techniques:

1. Direct Mapping: Main memory ka har block cache ke sirf aur sirf EK specific line mein map ho sakta hai. Formula: Cache Line = (Block Address) MOD (Total Cache Lines). Simple hardware, but conflict miss high hote hain.

2. Fully Associative Mapping: Main memory ka block cache ki KISI BHI empty line mein load ho sakta hai. Conflict miss zero, lekin tag comparison ke liye associative comparator hardware expensive hota hai.

3. K-way Set-Associative Mapping: Best trade-off! Cache ko sets mein divide kiya jata hai, har set mein K lines hoti hain. Block pehle set decide karta hai (Set = Block MOD Total Sets), phir us set ki kisi bhi K lines mein place ho sakta hai.

Performance Formulas:

- Hit Ratio (h): h = (Cache Hits) / (Total Memory Accesses)
- Miss Ratio: m = 1 - h
- Average Memory Access Time (AMAT): AMAT = T_{cache} + (1 - h) * T_{memory_penalty}

[!] MASTER QUICK REVISION SHEET (EXAM DAY FORMULA CAPSULE)

Exam se theek 1 ghante pehle revise karne ke liye Digital Fundamentals ke saare most-critical formulas, equations, aur direct rules ek jagah compile kiye gaye hain:

Topic	High-Priority Formula / Core Rule	CUET PG Exam Traps
Data Units	1 Byte = 8 bits; 1 Nibble = 4 bits; 1 KB = 1024 Bytes; 1 MB = 2^20 Bytes	Capital B = Byte, Small b = bit! Speed is in bps, Storage is in Bytes
Base Conversion	Binary <-> Octal: 3-bit groups; Binary <-> Hex: 4-bit groups	Fractional parts must be grouped from left-to-right (pad zeros on right!)
Complements	r's comp = (r-1)'s comp + 1. 2's comp = invert bits + 1	2's complement addition mein end carry DISCARD hota hai. 1's comp mein add back hota hai
2's Comp Range	n-bit signed range: -2^(n-1) to +(2^(n-1) - 1)	8-bit: -128 to +127. Zero is uniquely 00000000. Double zero does not exist
Floating Point	Single: 1 sign, 8 exp (bias 127), 23 fraction. Double: 1, 11 (bias 1023), 52	E=all 1s, F=0 is Infinity! E=all 1s, F!=0 is NaN! E=all 0s, F=0 is Zero
Boolean Laws	A + A'B = A + B; AB + A'C + BC = AB + A'C (Consensus); A + BC = (A+B)(A+C)	Dual: swap AND and OR, swap 0 and 1. Do NOT complement variables!
Universal Gates	NAND & NOR are universal. Min NAND for XOR = 4, for XNOR = 5	Min NOR for XOR = 5, for XNOR = 4! (Opposite for NOR!)
Full Adder	Sum = A XOR B XOR Cin; Cout = AB + Cin(A XOR B)	Built using: 2 Half Adders + 1 OR Gate.
Full Subtractor	Diff = A XOR B XOR Bin; Bout = A'B + Bin(A XOR B)'	Built using: 2 Half Subtractors + 1 OR Gate.
Multiplexer (MUX)	2^n inputs require n select lines. Acts as Universal Circuit.	4-to-1 MUX needs 2 select lines; 16-to-1 MUX needs 4 select lines
Flip-Flop Equations	SR: Q(t+1)=S+R'Q; JK: Q(t+1)=JQ'+K'Q; D: Q(t+1)=D; T: Q(t+1)=T XOR Q	JK when J=K=1 TOGGLES. Race-around occurs in level-triggered JK when tp > Delta_t
Shift Registers	SISO (n pulses to enter, n-1 to retrieve); PIPO fastest (1 pulse)	PISO = Parallel-in Serial-out; SIPO = Serial-in Parallel-out
Counters	Ring Counter: MOD = N states; Johnson Counter: MOD = 2N states	For 4 flip-flops: Ring has 4 states (12 unused); Johnson has 8 states (8 unused)!
Cache & Memory	DRAM requires periodic refresh (capacitors); SRAM does not (flip-flops)	AMAT = Tcache + (1 - h) * Tmemory. HDD and Tape are backup devices.

[MUST REMEMBER FOR CUET PG] Final Exam Tip by Antigravity

CUET PG mein speed aur accuracy dono critical hain. Truth table based questions mein directly options check karein (substitution method), K-Map mein sabse pehle Essential Prime Implicants identify karein, aur complement calculations mein binary inspection shortcut (copy till first 1, then flip) use karein.